Lab3
Machine Translation

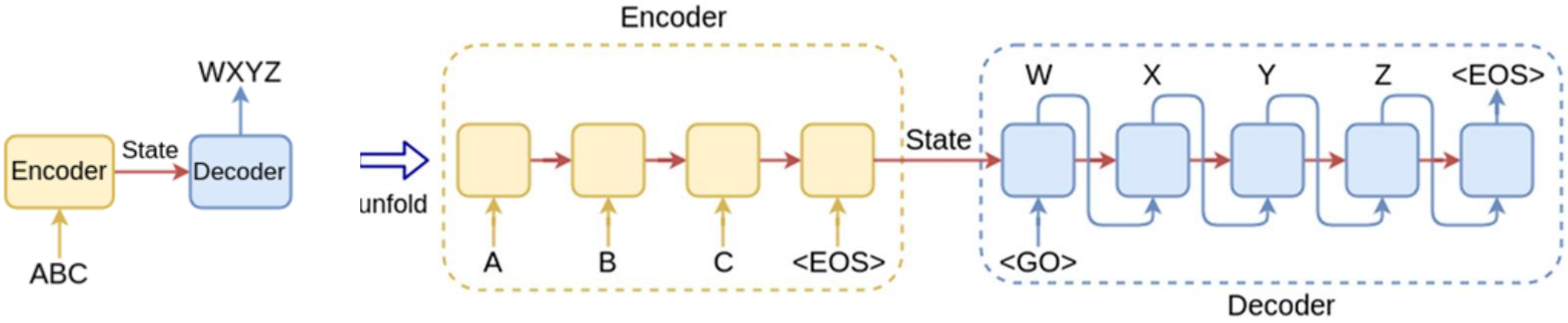
Dataset: English-Chinese Translation

- 20k-en-zh-translation-pinyin-hsk
  - https://huggingface.co/datasets/swaption2009/20k-en-zh-translation-pinyin-hsk

english	chinese
Slowly and not without struggle, America began...	美国缓慢地开始倾听，但并非没有艰难曲折。
Dithering is a technique that blends your colo...	抖动是关于颜色混合的技术，使你的作品看起来更圆滑，或者只是创作有趣的材质。
This paper discusses the petrologic characteri...	本文以珲春早第三纪含煤盆地的地质构造背景为依据，分析了煤系地层的岩石学特征。
The second encounter relates to my grandfather...	第二次事件跟我爷爷的宝贝匣子有关。
One way to address these challenges would be t...	解决这些挑战的途径包括依照麻瓜在南非的经验设立真相与和解委员会。
...	...

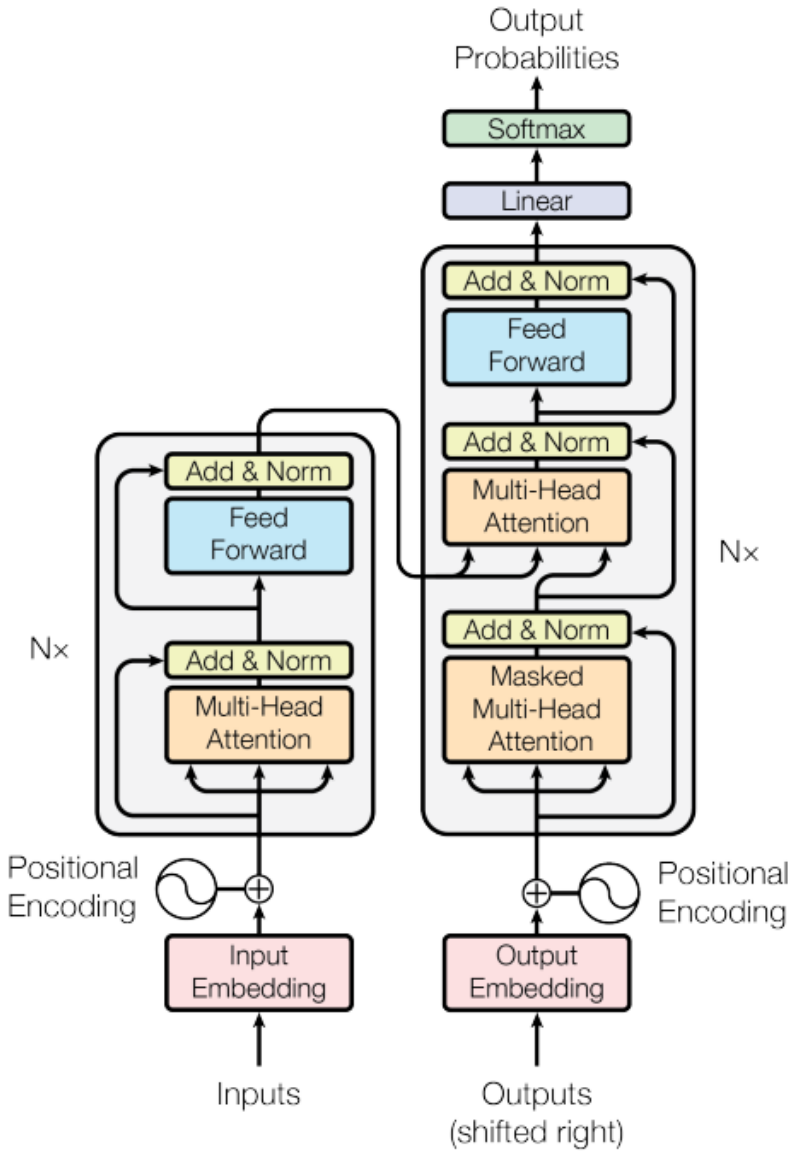
Seq2Seq Model

- Encoder-Decoder structure
- Applications: machine translation, chatbot…



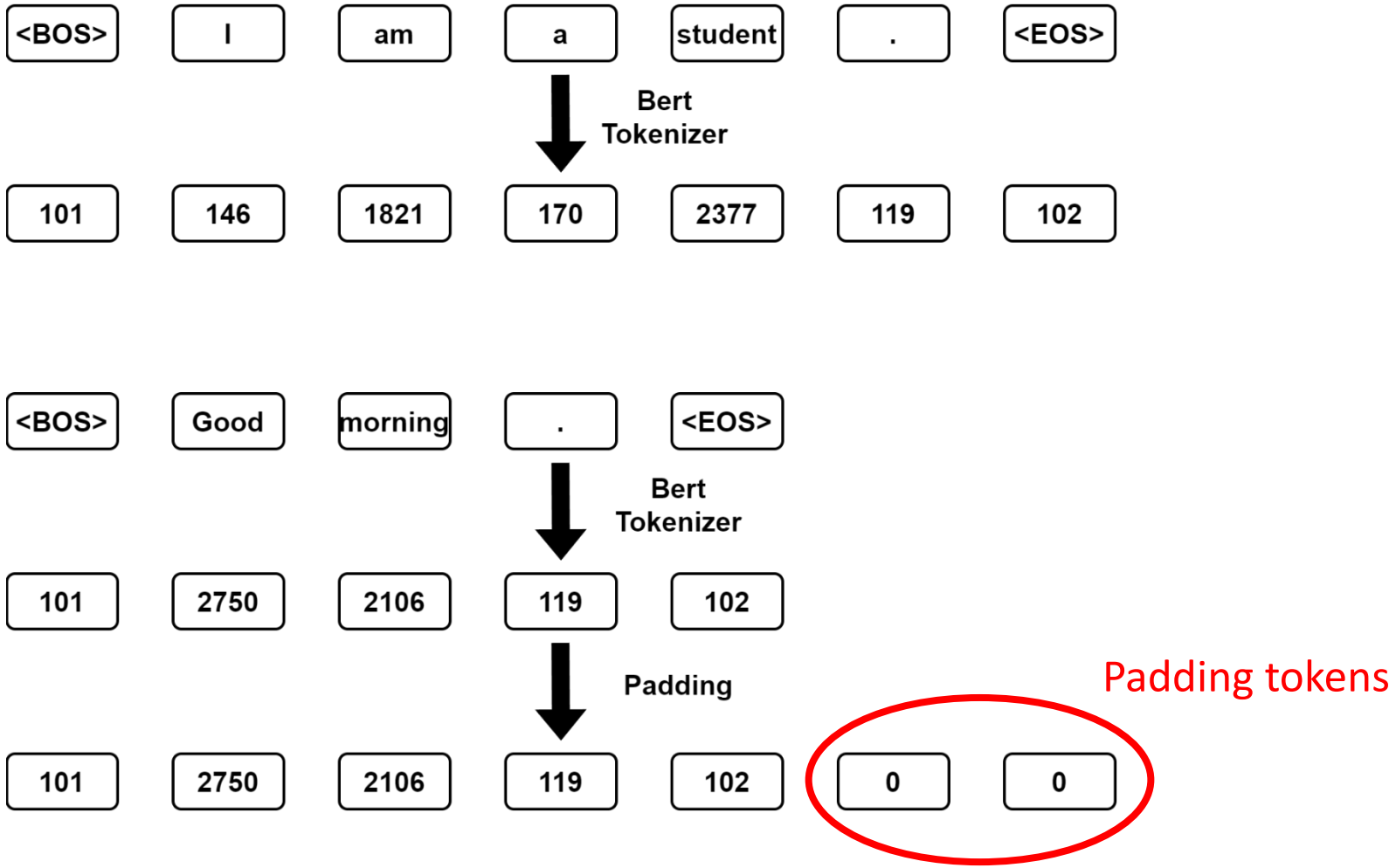
Transformer Model Architecture

- Embedding Layer
- Positional Encoding
- Encoder
  - Multi-head self attention
  - FFN
- Decoder
  - Masked multi-head self attention
  - Multi-head cross attention
  - FFN



Input Paddings

- Make the input sentence equal-length



Masks

- Masking is directly applied in attention score
- Future mask
  - Used in decoder self attention
  - Prevent from attending to the future tokens
- Padding mask
  - Used in encoder, decoder self attention both
  - Prevent from attending to the padding tokens

	<BOS>	I	am	a	student
<BOS>	-∞	-∞	-∞	-∞	-∞
I		-∞	-∞	-∞	-∞
am			-∞	-∞	-∞
a				-∞	-∞
student					-∞

<BOS>	Good	morning	.	<EOS>	<PAD>	<PAD>
					-∞	-∞

Tasks in this lab

1. In “Lab03.ipynb”
   - Build vanilla transformer (from Attention is All You Need) by yourself
     - https://arxiv.org/abs/1706.03762
   - You should build the encoder, decoder by yourself
   - Inside the encoder/decoder, you should build the attention mechanism (multi-head attention) by yourself

(You can’t call the model directly with the command)

```
self.encoder_layer = nn.TransformerEncoderLayer(d_model, nhead, d_feedforward)
self.encoder = nn.TransformerEncoder(self.encoder_layer, num_enc_layer)
self.decoder_layer = nn.TransformerDecoderLayer(d_model, nhead, d_feedforward)
self.decoder = nn.TransformerDecoder(self.decoder_layer, num_dec_layer)

self.transformer = nn.Transformer(d_model, nhead, num_enc_layer, num_dec_layer, d_feedforward, dropout)

self.attention = nn.MultiheadAttention(emb_dim, nhead)
```

Tasks in this lab

1. In “Lab03.ipynb”
   - The parameter size of transformer encoder block should be less than **100M** !
     (put the screenshot in your report)
   - Achieve at least **0.18** validation accuracy (BLEU score)
     (put the screenshot in your report)

Tasks in this lab

1. In “Lab03.ipynb”
   - You can NOT use the following models, TA will check your code!
     - torch.nn.TransformerEncoderLayer
     - torch.nn.TransformerEncoder
     - torch.nn.TransformerDecoderLayer
     - torch.nn.TransformerDecoder
     - torch.nn.Transformer
     - torch.nn.MultiheadAttention

Code

```
EMB_SIZE = 128
NHEAD = 8
FFN_HID_DIM = 128
NUM_ENCODER_LAYERS = 1
NUM_DECODER_LAYERS = 1
SRC_VOCAB_SIZE = tokenizer_cn.vocab_size
TGT_VOCAB_SIZE = tokenizer_en.vocab_size
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

seq2seq_network = Seq2SeqNetwork(NUM_ENCODER_LAYERS, NUM_DECODER_LAYERS, EMB_SIZE,
                                 NHEAD, SRC_VOCAB_SIZE, TGT_VOCAB_SIZE, FFN_HID_DIM)

for p in seq2seq_network.parameters():
    if p.dim() > 1:
        nn.init.xavier_uniform_(p)

seq2seq_network = seq2seq_network.to(DEVICE)
param_seq2seq_network = sum(p.numel() for p in seq2seq_network.parameters())
print (f"The parameter size of model is {param_seq2seq_network/1000} k")
```

You can set whatever you want

Hint：The larger isn't always the better.

```
The parameter size of model is 10421.828 k
```

The parameter size of transformer encoder block should be less than **100M** !

Code

```
from timeit import default_timer as timer
transformer = transformer.to(DEVICE)

best_acc = 0
for epoch in range(1, NUM_EPOCHS+1):
    start_time = timer()
    train_loss = train_epoch(transformer, optimizer, cn_to_en_train_loader)
    end_time = timer()
    val_loss, val_acc = evaluate(transformer, cn_to_en_test_loader)

    print((f"Epoch: {epoch}, Train loss: {train_loss:.3f}, Val loss: {val_loss:.3f},


    # Save the best model so far.
    if val_acc > best_acc:
        best_acc = val_acc
        best_state_dict = transformer.state_dict()
        torch.save(best_state_dict, MODEL_SAVE_PATH)
        print("(model saved)")
```

```
Epoch: 7, Train loss: 4.775, Val loss: 5.109, Val Acc: 0.188, Epoch time = 39.070s
(model saved)
Epoch: 8, Train loss: 4.590, Val loss: 5.036, Val Acc: 0.200, Epoch time = 39.189s
(model saved)
Epoch: 9, Train loss: 4.412, Val loss: 4.994, Val Acc: 0.210, Epoch time = 39.269s
(model saved)
```

Achieve at least 0.18 BLEU score (validation accuracy)

Tasks in this lab

2. Write a report

- Required
  - Screenshot of task-1
    - Parameter size
    - Accuracy (BLEU score)
  - In task-1
    - The structure of Transformer you implement (ex. number of encoder layers, decoder layers)
    - Training strategy (ex. learning rate scheduler)
  - Anything you do to improve the performance.
  - Any difficulty you encounter

- **Plagiarism is forbidden !!!**
- **There should be comment in your code**
- **You should make sure your code is executable**

Score

- Two requirements in Task-1 are fulfilled (50%)
  - Parameter size < 100M
  - Validation accuracy (BLEU score) > 0.18
- Report (30%)
- Performance (20%)
  - Performance rank for Task-1

Reminder

- Submit Deadline : 2 week (2024/10/21 23:59 PM)
- Please compress a folder into a zip file named StudentID.zip (example: 313555555.zip) and upload it to new e3
- The folder should include:
  - Lab3.ipynb
  - Model.ckpt
  - StudentID_report.pdf (example: 313555555_report.pdf)

HAVE FUN !!!